HCCL算子开发入门（AICPU_TS引擎）



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V5.0	2026/8/30	张兴平	全部	初版内容

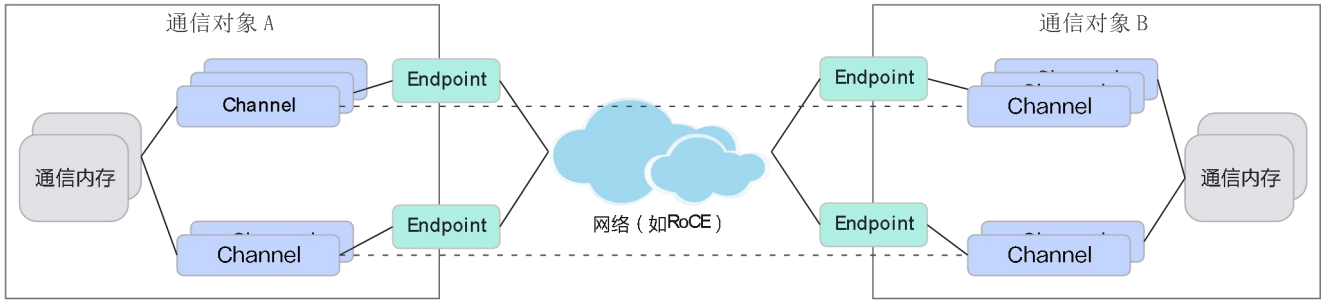
目录

Part 1 HCCL算子编程模型和API回顾

Part 2 HCCL算子开发-AICPU+TS引擎

HCCL编程模型总结

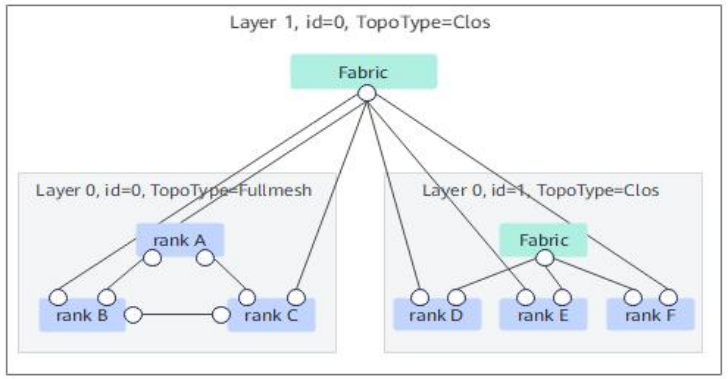
通信模型



Channel：

- 本端通信对象基于特定Endpoint与远端通信对象特定Endpoint之间建立的通信通道；
- 用户使用Channel可以读写远端通信对象内存或与远端通信对象进行同步。

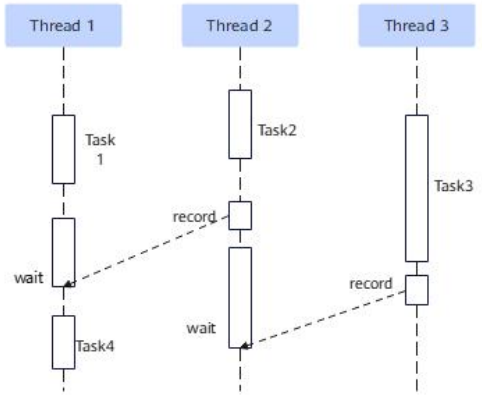
拓扑模型



- 以**RankGraph**的方式对分级拓扑进行建模，并提供查询接口

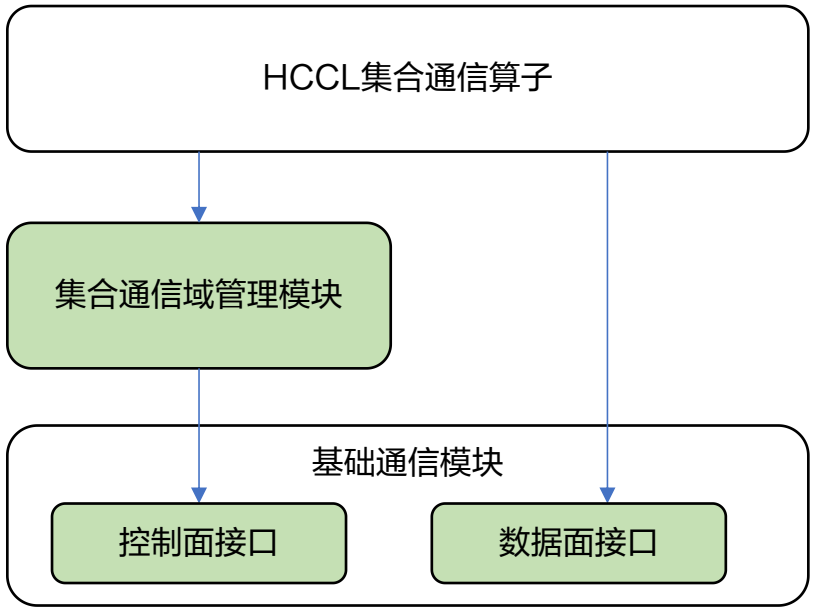
https://gitcode.com/cann

并发模型



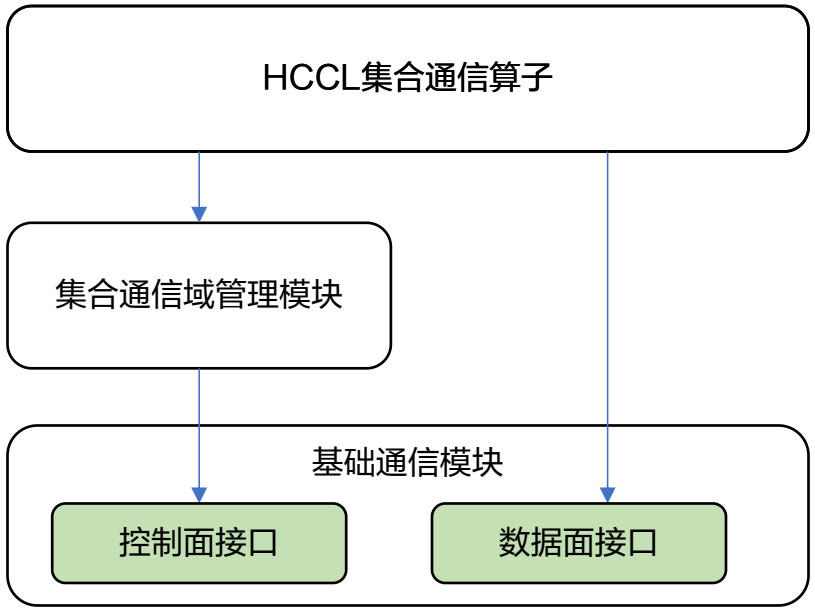
- 并发单元抽象为**Thread**，通信任务与Thread绑定，不同Thread上的任务可并发执行
- Thread包含多个**Notify**，Thread间可以通过Notify进行同步；

通信算子编程API层次



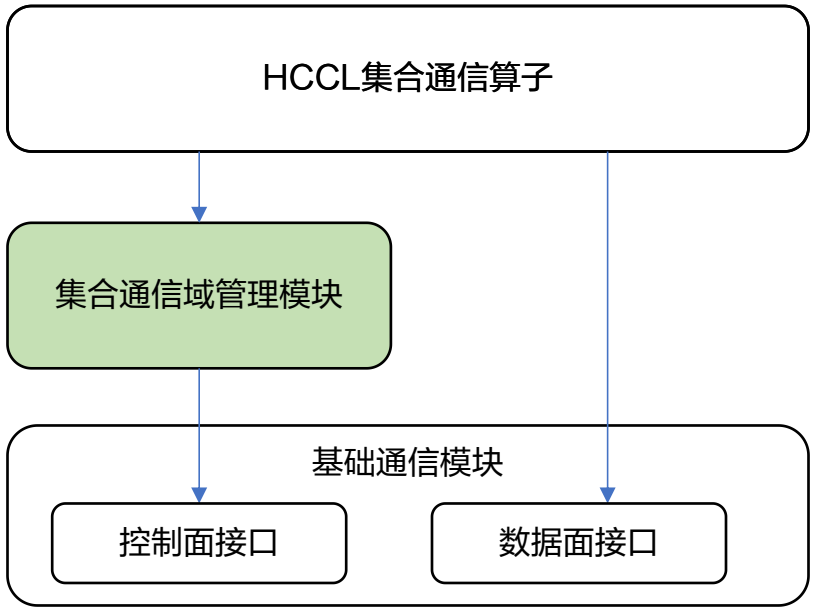
CANN

HCCL算子开发：通信算子编程API



类型		API	说明
控制面接口		HcommEndpointCreate	创建Endpoint
		HcommEndpointDestroy	销毁Endpoint
		HcommMemReg	内存注册
		HcommMemUnreg	内存解注册
		HcommMemExport	导出注册内存，用于交换
		HcommMemImport	导入远端交换的注册内存
		HcommChannelCreate	创建Channel
		HcommChannelDestroy	销毁Channel
		HcommThreadAlloc	申请Thread资源
		HcommThreadFree	释放Thread资源
数据面接口	本地操作	HcommLocalCopyOnThread	本地拷贝
		HcommLocalReduceOnThread	本地规约操作
		HcommThreadNotifyRecordOnThread	向其他Thread发送同步信号
		HcommThreadNotifyWaitOnThread	等待同步信号
	通信操作	HcommWriteOnThread	写远端内存操作
		HcommWriteReduceOnThread	写远端内存并做规约操作
		HcommReadOnThread	读远端内存操作
		HcommReadReduceOnThread	读远端内存并与本地做规约操作
		HcommChannelNotifyRecordOnThread	向远端发送同步信号
		HcommChannelNotifyWaitOnThread	等待远端同步信号
		HcommWriteWithNotifyOnThread	写远端内存并发送同步信号
		HcommWriteReduceWithNotifyOnThread	写远端内存并做规约操作，同时发送同步信号

HCCL算子开发：通信算子编程API



类型	API	说明
拓扑	HcclRankGraphGetLayers	查询拓扑层级信息。
	HcclRankGraphGetRanksByLayer	查询本rank所在拓扑实例的rank信息
	HcclRankGraphGetRankSizeByLayer	查询本rank在特定拓扑层级下的rank数量
	HcclRankGraphGetTopoTypeByLayer	查询本rank在特定拓扑层级下的拓扑类型
	HcclRankGraphGetInstSizeListByLayer	查询给定拓扑层级下的拓扑实例信息
	HcclRankGraphGetLinks	查询特定拓扑层级下源rank和目的rank之间的连接信息
Thread	HcclThreadAcquire	基于通信域获取Thread
	HcclThreadAcquireWithStream	
Channel	HcclChannelAcquire	基于通信域获取Channel
	HcclChannelGetHcclBuffer	基于Channel获取远端HCCL Buffer
通信上下文	HcclEngineCtxCreate	创建通信上下文资源
	HcclEngineCtxGet	查询通信上下文资源
	HcclEngineCtxCopy	将Host侧数据拷贝至通信上下文中
	HcclEngineCtxDestroy	销毁通信上下文资源
通信内存	HcclGetHcclBuffer	基于通信域获取HCCL Buffer
	HcclCommMemReg	基于通信域注册内存
	HcclChannelGetRemoteMems	获取Channel中交换的远端内存

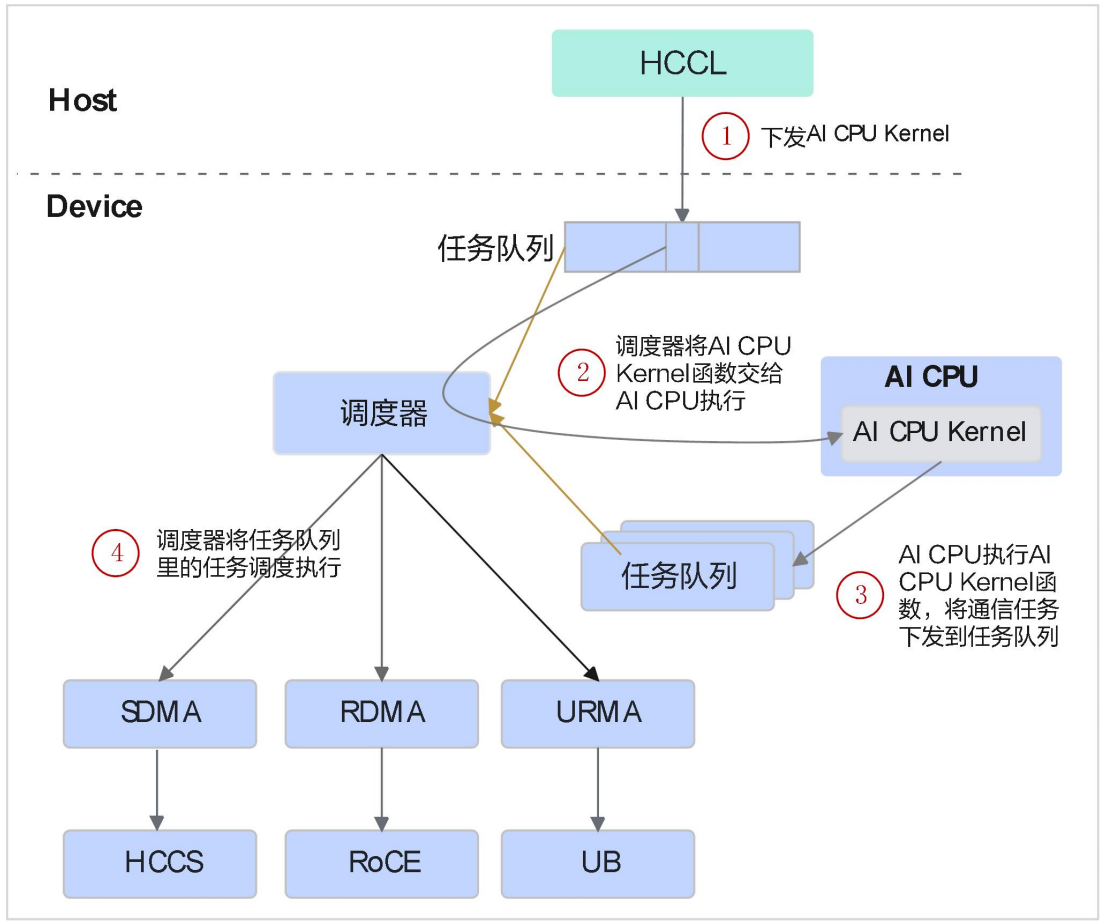
目录

Part 1 HCCL算子编程模型和API回顾

Part 2 HCCL算子开发-AICPU+TS引擎

HCCL算子执行流程：AICPU + TS通信引擎

AICPU+调度器(TS) 通信引擎算子执行流程



1. Host侧部分：

- 申请资源（Channel、Thread等）；
- 创建算子上下文缓存，缓存资源句柄；
- 下发AICPU Kernel任务至Device任务队列。

2. Device侧流程：

- AICPU Kernel被任务调度器调度给AI CPU执行；
- AICPU执行算法逻辑，将通信、同步等任务下发至其他任务队列；
- AI CPU提交的通信任务被调度器调度至执行器执行。

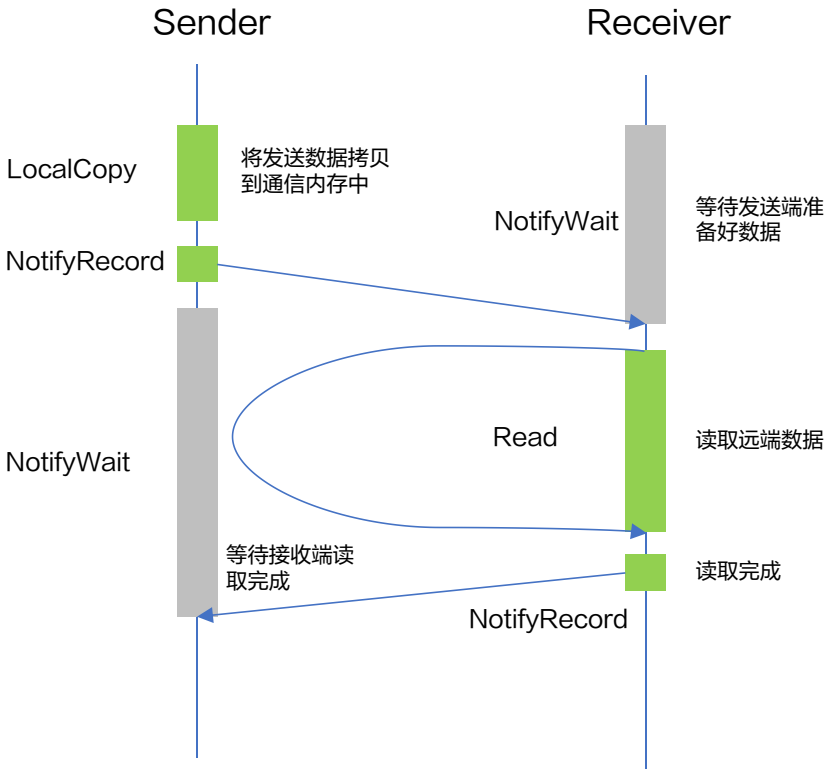
回顾：使用硬件调度器的好处？

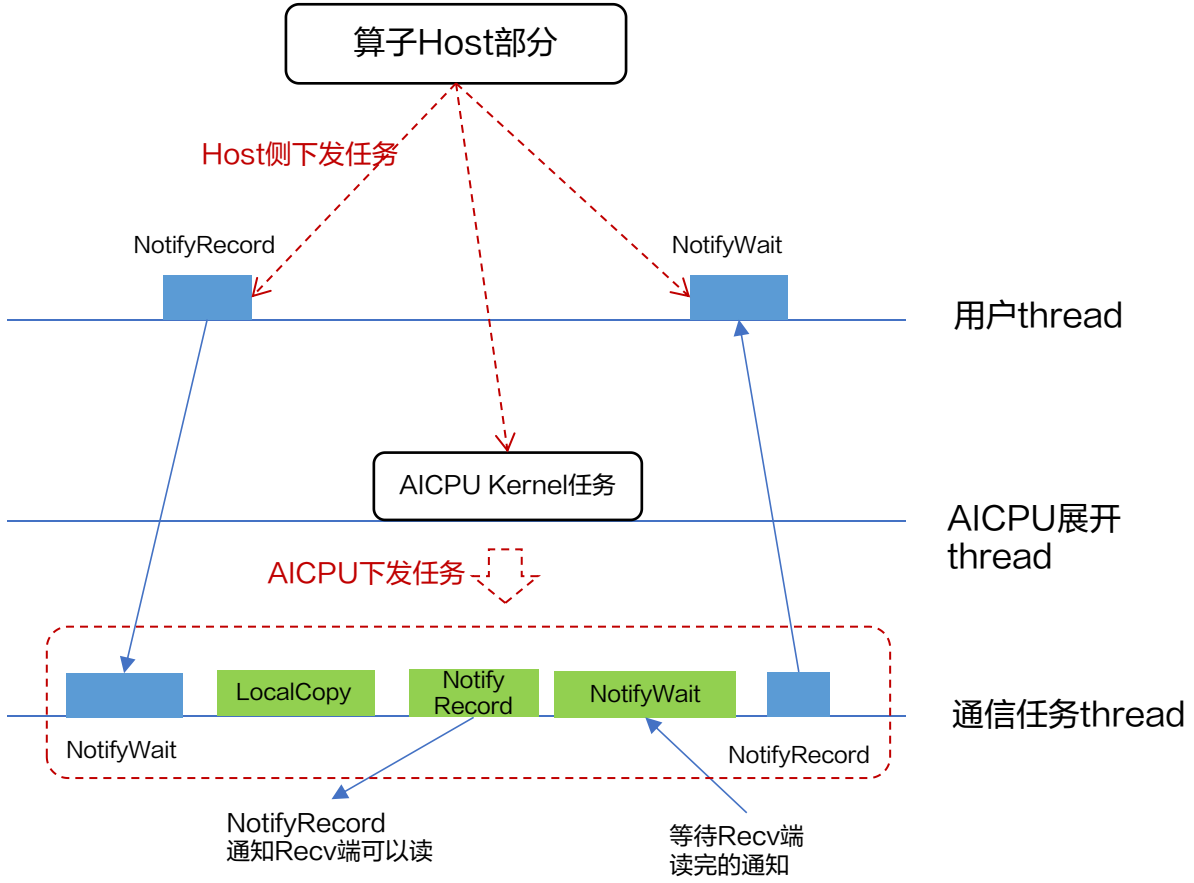
答：系统抖动较小，调度时延稳定

CANN

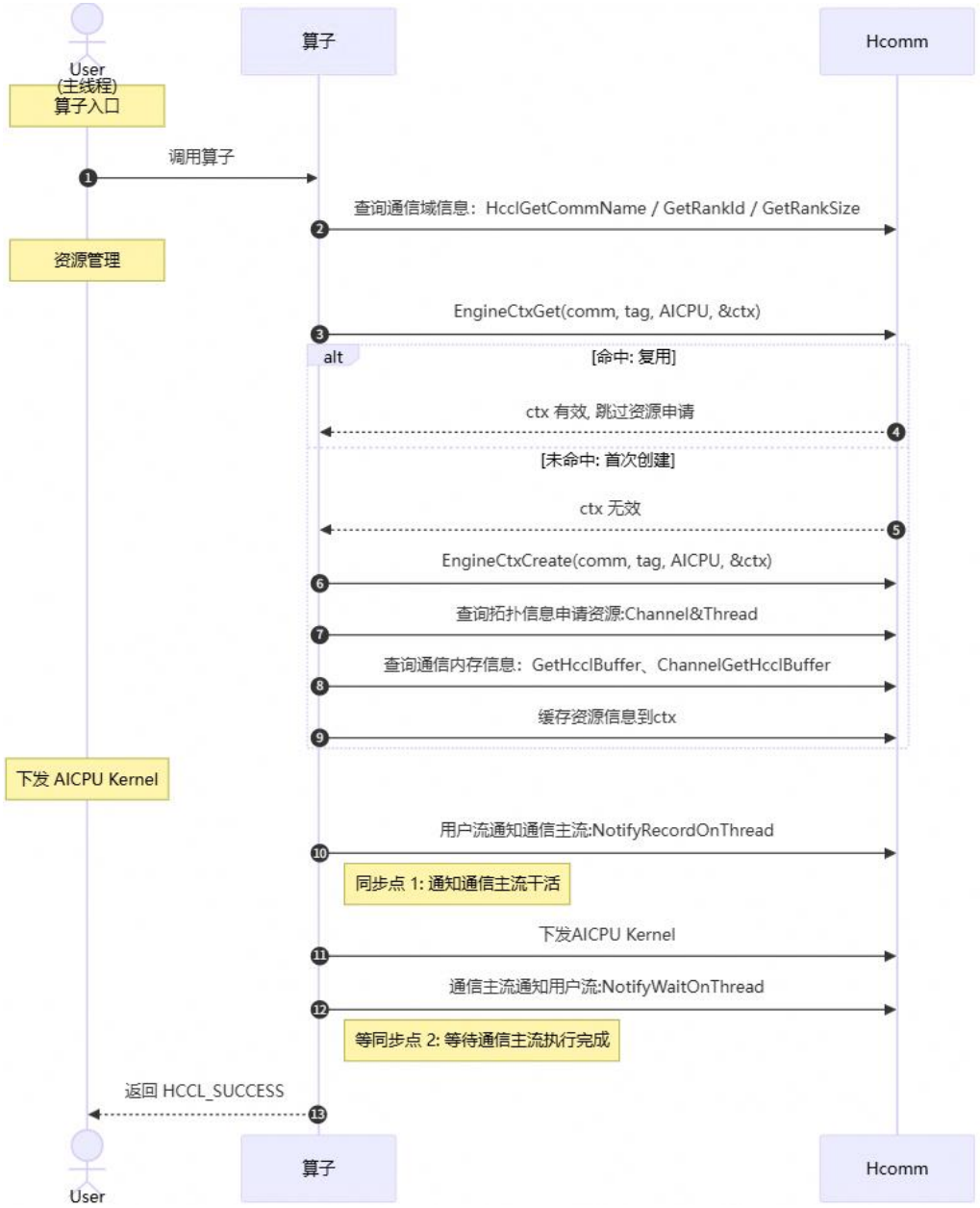
HCCL算子开发介绍：自定义点到点通信样例

- HcclSendCustom(sendBuf, count, dataType, destRank, comm, stream)
- HcclRecvCustom(recvBuf, count, dataType, srcRank, comm, stream)





HCCL算子开发介绍：自定义点到点通信样例（host侧流程）



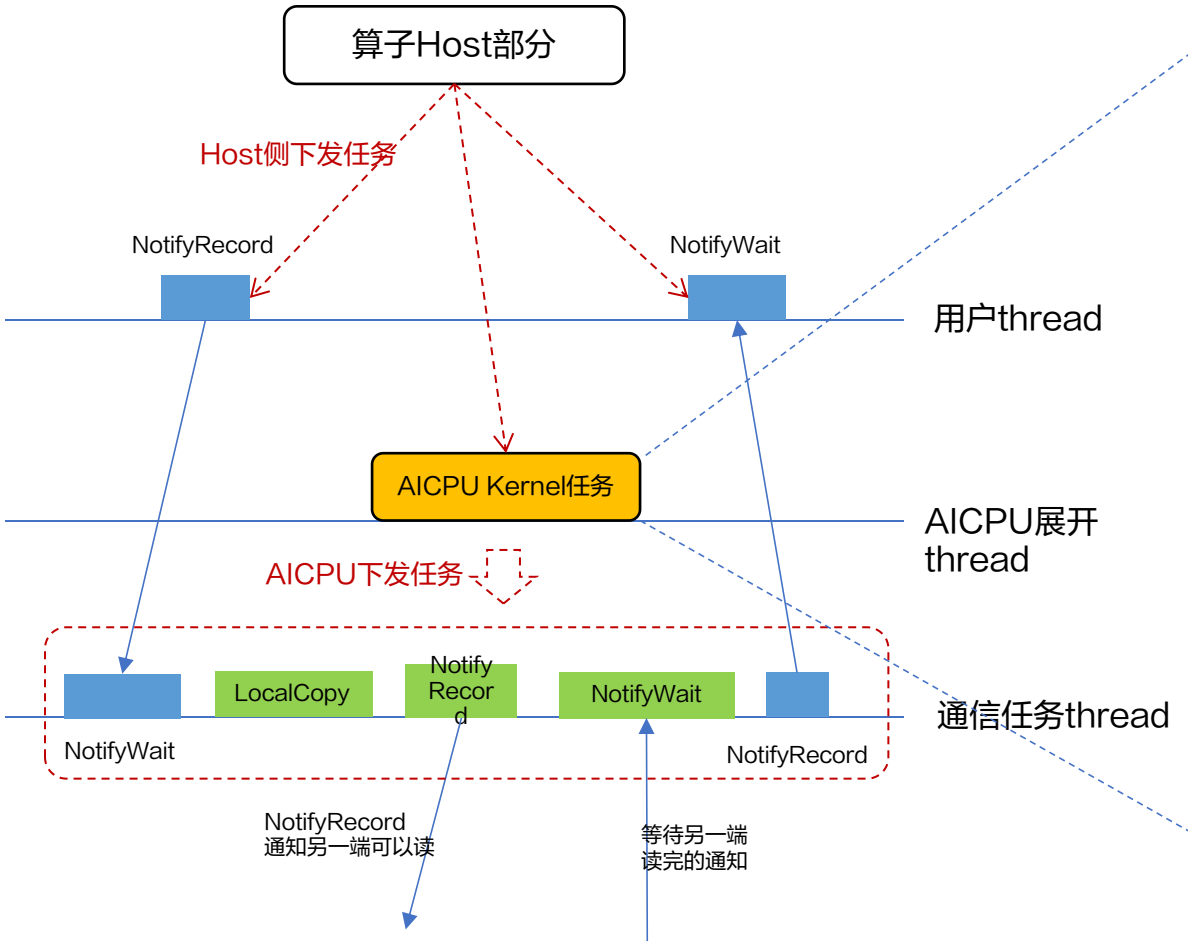

```cpp
HcclResult HcclSendCustom(void *sendBuf, uint64_t count, HcclDataType dtype, uint32_t destRank,
HcclComm comm, aclrtStream stream)
{
  // 准备 OpParam
  OpParam param;
    // 查询通信域信息
  HcclGetRankId  (comm, &rank);
  HcclGetRankSize (comm, &rankSize);
  // 资源管理: 按 tag 缓存 AlgResourceCtx
  void *ctx = nullptr;
  HcclResult rc = HcclEngineCtxGet(comm, param.tag, CommEngine::COMM_ENGINE_AICPU, &ctx, &size);
  if (rc != HCCL_SUCCESS) {
    HcclEngineCtxCreate(comm, param.tag, CommEngine::COMM_ENGINE_AICPU, size, &ctx);
    // 申请 Thread
    HcclThreadAcquireWithStream(comm, COMM_ENGINE_CPU_TS, stream, 1 /*notifyNum*/, &cpuThread);
    HcclThreadAcquire(comm, COMM_ENGINE_AICPU_TS, 1 /*threadNum*/, 1 /*notifyNum*/, &aicpuThread);
    // 建立 Channel
    HcclChannelDesc desc;
    desc.remoteRank    = destRank;
    ...
    HcclChannelAcquire(comm, CommEngine::COMM_ENGINE_AICPU, &desc, 1, &channelHandle);
    // 查询通信 Buffer
    HcclGetHcclBuffer    (comm, &localBuffer.addr, &localBuffer.size);
    HcclChannelGetHcclBuffer (comm, channelHandle, &remoteBuffer.addr, &remoteBuffer.size);
    // ctx 拷到 device 侧
    aclrtMemcpy(ctx, size, &resCtxHost, size, ACL_MEMCPY_HOST_TO_DEVICE);
  }
  param.resCtx = (AlgResourceCtx *)ctx;
    // Host stream 通知 AICPU 主 thread 开工

  HcclAicpuKernelLaunch(comm, func, args, …, stream);
  return HCCL_SUCCESS;
}
```

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/04_custom_ops_p2p/op_host/send.cc

CANN

HCCL算子开发介绍：自定义点到点通信样例（device侧流程）

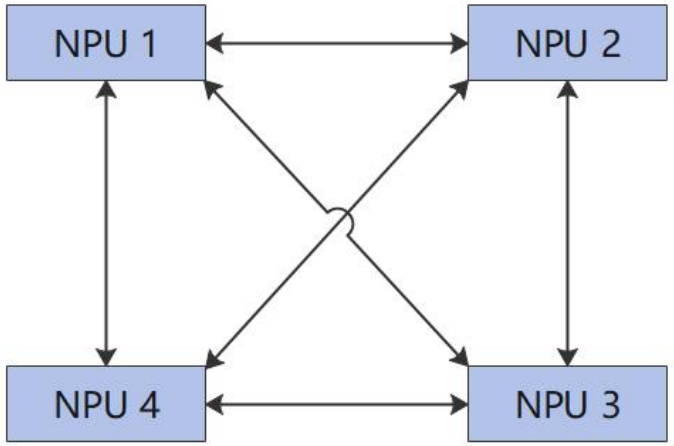



```cpp
int HcclP2PAicpuKernel(OpParam *param) {
  // 准备: 占 comm + 批模式 + 等 用户流同步
  HcommAcquireComm(param->commName);
  HcommBatchModeStart(param->tag);
  HcommThreadNotifyWaitOnThread(param->resCtx->aicpuThread, 0, timeout);
  // ----------------------------------------------------------------
  // 算法编排
  // ----------------------------------------------------------------
  uint64_t size = param->count * SIZE_TABLE[param->dataType];
  if (param->opType == HcclCMDType::HCCL_CMD_SEND) {
    HcommLocalCopyOnThread    (aicpuThread, localBuffer.addr, inputPtr, size);
    HcommChannelNotifyRecordOnThread(aicpuThread, channel, NOTIFY_IDX_ACK);
    HcommChannelNotifyWaitOnThread (aicpuThread, channel, NOTIFY_IDX_DATA_SIGNAL, 1836);
  } else { // RECEIVE
    HcommChannelNotifyWaitOnThread (aicpuThread, channel, NOTIFY_IDX_ACK, 1836);
    HcommReadOnThread(aicpuThread, channel, outputPtr, remoteBuffer.addr, size);
    HcommChannelNotifyRecordOnThread(aicpuThread, channel, NOTIFY_IDX_DATA_SIGNAL);
  }
  // 通知用户流
  HcommThreadNotifyRecordOnThread(aicpuThread, cpuThreadOnAicpu, 0);
  HcommBatchModeEnd(param->tag);
  HcommReleaseComm(param->commName);
  return 0;
}
```

参考链接：
https://gitcode.com/cann/hccl/blob/master/examples/04_custom_ops_p2p/op_kernel_aicpu/exec_op.cc

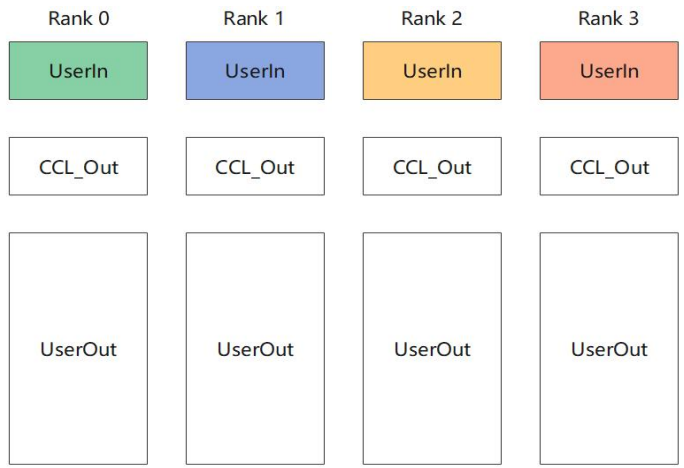
HCCL算子开发介绍 – 更复杂的例子

AllGather算子

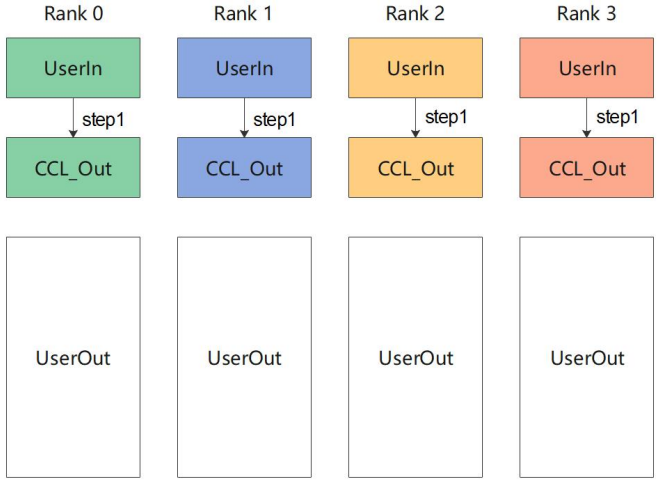


假定通信域中有4个NPU，每个NPU与另外三个NPU都存在物理链路

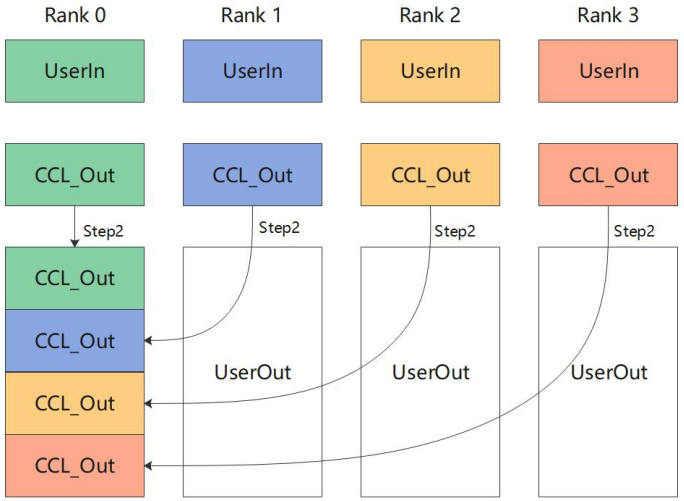
思考：用什么算法比较好?



初始状态



Step1



Step2

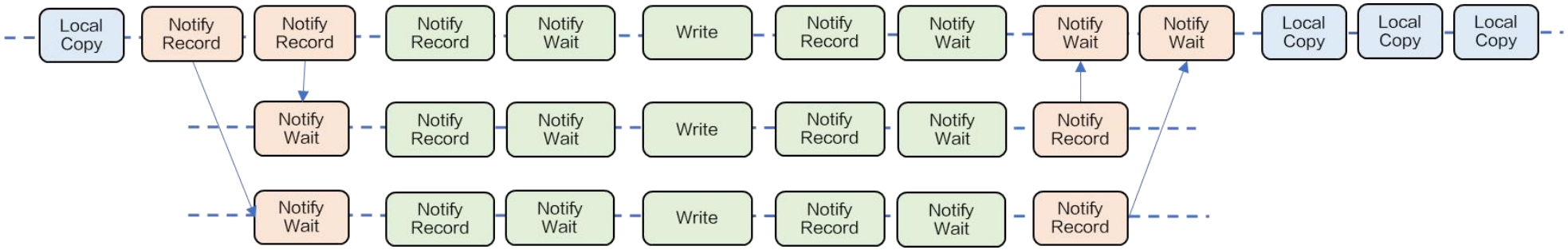
（备注：其它rank也一样）

Step2中，每个rank都将自己CCL Buffer中的数据通过不同的直连物理链路发往其它所有rank，最大程度利用通信带宽。

由于链路不复用，发往其它3个不同rank的数据可以**使用3个thread并行发送，提升通信算子性能。**

CANN

HCCL算子开发介绍 – 更复杂的例子




```
// 按 ccl buffer 大小，每 loop 处理一个 slice
sliceSize    = min(maxDataCountPerLoop, 剩余) * dtypeSize;
cclBuffBase  = cclMem.addr + sliceSize * myRank;   // 本端我这块的位置
for (loop ...) {
    // ① 本地: inputPtr → cclMem[myRank 块]
    HcommLocalCopyOnThread(threads[0], cclBuffBase, inputPtr+off, sliceSize);
    // ② 前同步: 主 thread 通知所有从 thread 开工
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyRecordOnThread(threads[0], threads[i], 0);
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyWaitOnThread  (threads[i], 0, 1836);
    // ③ 数据交换: threads[i] 跟远端 rank i 做 ACK + Write(单边写) + DATA
    for (i = 0; i < threads.size(); i++) {
        ch          = channels[i].handle;            // 与远端 rank i 的 channel
        remoteCclBuf= channels[i].remoteCclMem.addr
                    + sliceSize * myRank;            // 写到对端 cclMem 的我这块
        HcommChannelNotifyRecordOnThread(threads[i], ch, NOTIFY_IDX_ACK);
        HcommChannelNotifyWaitOnThread  (threads[i], ch, NOTIFY_IDX_ACK, 1836);
        //          ──────  ACK 握手: 双方 staging 就绪
        HcommWriteOnThread(threads[i], ch, remoteCclBuf, cclBuffBase, sliceSize);
        //          ──────  单边写: 本端 → 远端 cclMem
        HcommChannelNotifyRecordOnThread(threads[i], ch, NOTIFY_IDX_DATA_SIGNAL);
        HcommChannelNotifyWaitOnThread  (threads[i], ch, NOTIFY_IDX_DATA_SIGNAL, 1836);
        //          ──────  DATA 握手: 双方写完
    }
    // ④ 后同步: 从 thread 通知主 thread 收齐
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyWaitOnThread  (threads[i], i, 1836);
    for (i = 1; i < threads.size(); i++)
        HcommThreadNotifyRecordOnThread(threads[i], threads[0], i-1);
    // ⑤ 拷到 output: output[rankId*dataSize+off] ← cclMem[rankId 块]
    for (rankId = 0; rankId < rankSize; rankId++) {
        HcommLocalCopyOnThread(threads[0], outputPtr + rankId*dataSize + off, cclMem.addr + rankId*sliceSize, sliceSize);
    }
}
```

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/aicpu/op_kernel_aicpu/exec_op.cc

HCCL算子开发最佳实践：模块调用关系

Step1-算法选择：

拓扑解析：通过CANN/hcomm的**通信域拓扑查询API**，感知当前集群拓扑和硬件形态；

算法选择：根据当前**集群拓扑**和**硬件形态**，选择最佳性能的集合通信算法（Mesh、Ring、NHR等）。

Step2-算法资源准备：

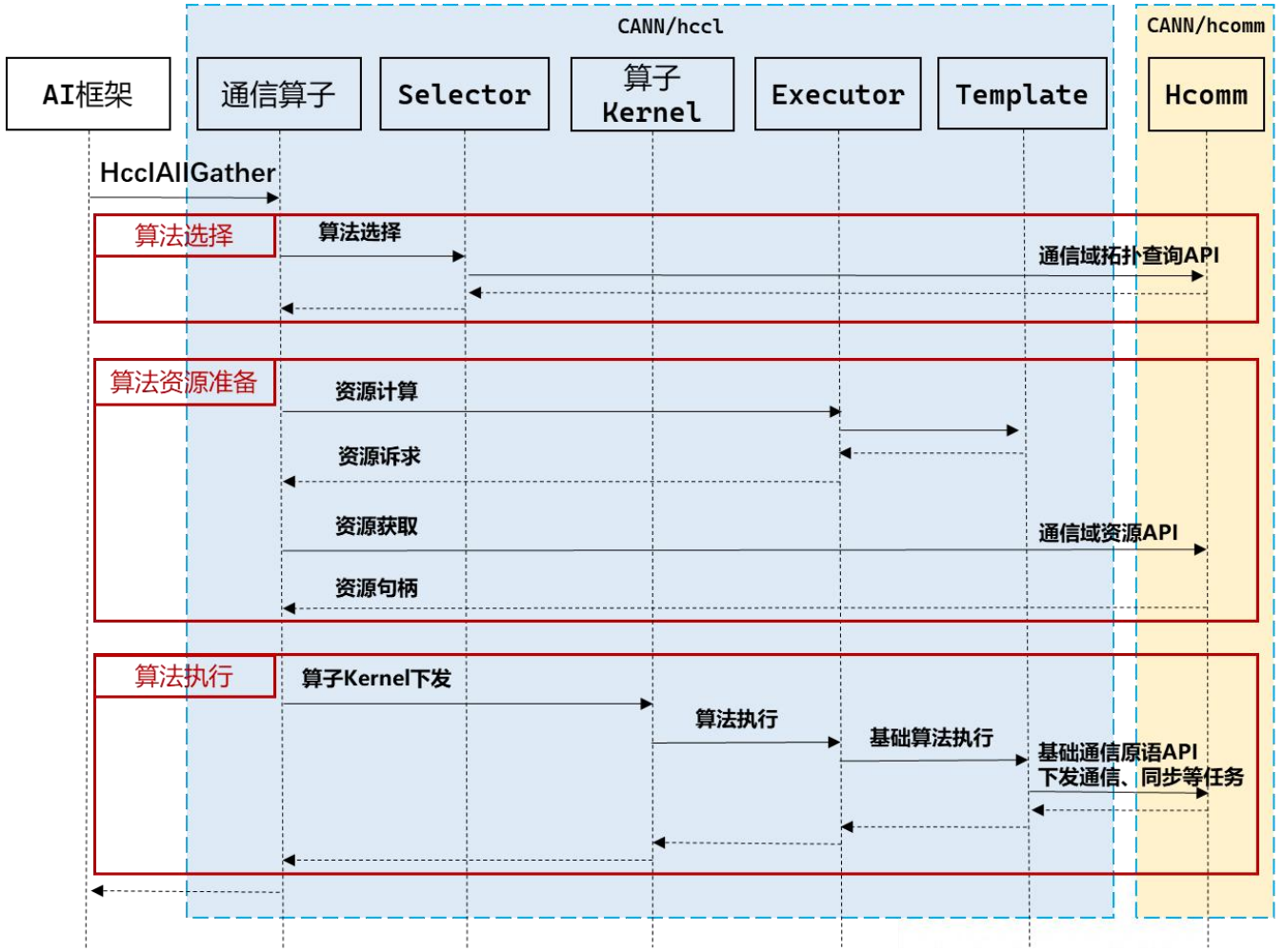
资源计算：依据前一步选择的**算法**以及**拓扑信息**，计算当前算法的通信资源诉求（channel、thread、notify等资源）；

资源获取：根据当前算法资源诉求，通过CANN/hcomm的**通信域资源API**获取对应资源句柄

Step3-算法执行：

Kernel下发： 提交当前算子的AI CPU Kernel至任务队列

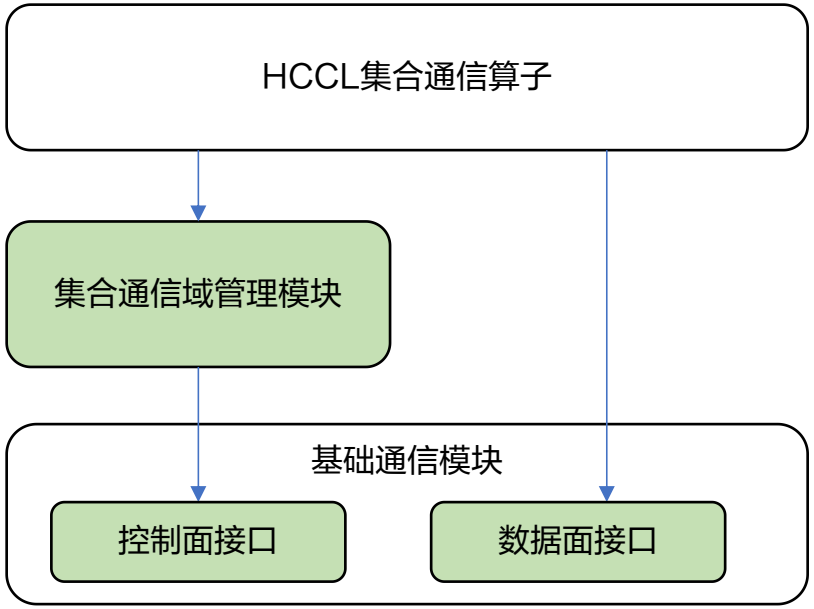
算法执行：使用创建好的资源，根据当前算法所需步骤，依次调用CANN/hcomm的**基础通信原语API**进行任务下发。



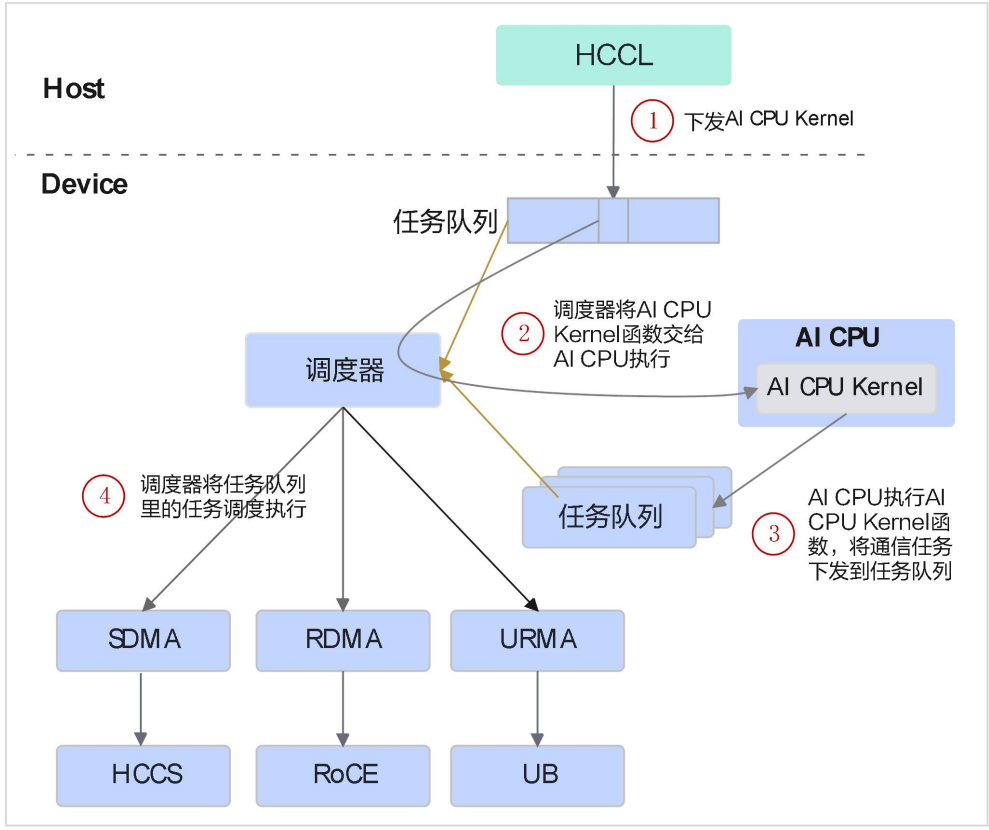
CANN

课程小结

- **HCCL算子编程API**



- **AICPU+TS引擎的通信算子开发**



CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN